Create a Map Reduce program to sort the content in an alphabetic order listing only top 10 maximum occurrences of words.

Driver:

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.GenericOptionsParser;

public class TopN {
public static void main(String[] args) throws Exception {
Configuration conf = new Configuration();
String[] otherArgs = (new GenericOptionsParser(conf, args)).getRemainingArgs();
if (otherArgs.length != 2) {
System.err.println("Usage: TopN <in> <out>");
System.exit(2);
}
Job job = Job.getInstance(conf);
job.setJobName("Top N");
job.setJarByClass(TopN.class);
job.setMapperClass(TopNMapper.class);
job.setReducerClass(TopNReducer.class);
job.setOutputKeyClass(Text.class);

job.setOutputValueClass(IntWritable.class);
FileInputFormat.addInputPath(job, new Path(otherArgs[0]));
FileOutputFormat.setOutputPath(job, new Path(otherArgs[1]));
System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

Mapper

```
public static class TopNMapper extends Mapper<Object, Text, Text, IntWritable> {

private static final IntWritable one = new IntWritable(1);

private Text word = new Text();

private String tokens = "[_|$#<>\\^=\\[\\]\\*/\\\\,;,.\\-:()?!\"']";

public void map(Object key, Text value, Mapper<Object, Text, Text,

IntWritable>.Context context) throws IOException, InterruptedException {

String cleanLine = value.toString().toLowerCase().replaceAll(this.tokens, " ");

StringTokenizer itr = new StringTokenizer(cleanLine);

while (itr.hasMoreTokens()) {
this.word.set(itr.nextToken().trim());
context.write(this.word, one);
}
}
}
}
```

Combiner:

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
public class TopNCombiner extends Reducer<Text, IntWritable, Text, IntWritable> {
public void reduce(Text key, Iterable<IntWritable> values, Reducer<Text, IntWritable, Text,
IntWritable>.Context context) throws IOException, InterruptedException {
int sum = 0;
for (IntWritable val : values)
sum += val.get();
context.write(key, new IntWritable(sum));
}
}
```

Reducer:

```
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;
```

```
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
import utils.MiscUtils;
public class TopNReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
private Map<Text, IntWritable> countMap = new HashMap<>();
public void reduce(Text key, Iterable<IntWritable> values, Reducer<Text, IntWritable, Text,
IntWritable>.Context context) throws IOException, InterruptedException {
int sum = 0;
for (IntWritable val : values)
sum += val.get();
this.countMap.put(new Text(key), new IntWritable(sum));
}
protected void cleanup(Reducer<Text, IntWritable, Text, IntWritable>.Context context) throws
IOException, InterruptedException {
Map<Text, IntWritable> sortedMap = MiscUtils.sortByValues(this.countMap);
int counter = 0;
for (Text key : sortedMap.keySet()) {
if (counter++ == 20)
break;
context.write(key, sortedMap.get(key));
}
}
}
```

MiscUtils.java

```
package utils;
import java.util.*;
public class MiscUtils {
public static <K extends Comparable, V extends Comparable> Map<K, V> sortByValues(Map<K,
V> map) {
List<Map.Entry<K, V>> entries = new LinkedList<Map.Entry<K, V>>(map.entrySet());
Collections.sort(entries, new Comparator<Map.Entry<K, V>>() {
@Override
public int compare(Map.Entry<K, V> o1, Map.Entry<K, V> o2) {
return o2.getValue().compareTo(o1.getValue());
}
});
//LinkedHashMap will keep the keys in the order they are inserted
//which is currently sorted on natural ordering
Map<K, V> sortedMap = new LinkedHashMap<K, V>();
for (Map.Entry<K, V> entry : entries) {
sortedMap.put(entry.getKey(), entry.getValue());
}
return sortedMap;
}
}
```

```
root@hp:~# mkdir topn
cd topn

mkdir -p utils
root@hp:~/topn# nano TopN.java
root@hp:~/topn# nano TopNMapper.java
root@hp:~/topn# nano TopNCombiner.java
root@hp:~/topn# nano TopNReducer.java
root@hp:~/topn# nano utils/MiscUtils.java
root@hp:~/topn# javac -classpath `hadoop classpath` -d . TopN.java TopNMapper.java TopNReducer.java TopNCombiner.java utils/MiscUtils.java
Note: utils/MiscUtils.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
root@hp:~/topn# javac -classpath `hadoop classpath` -d . TopN.java TopNMapper.java TopNReducer.java TopNCombiner.java utils/MiscUtils.java
Note: utils/MiscUtils.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
root@hp:~/topn# jar -cvf topn.jar *
added manifest
adding: TopN.class(in = 1550) (out= 873)(deflated 43%)
adding: TopN.java(in = 1120) (out= 442)(deflated 60%)
adding: TopNCombiner.class(in = 1594) (out= 669)(deflated 58%)
adding: TopNCombiner.java(in = 529) (out= 256)(deflated 51%)
adding: TopNMapper.class(in = 1958) (out= 894)(deflated 54%)
adding: TopNMapper.java(in = 832) (out= 417)(deflated 49%)
adding: TopNReducer.class(in = 2546) (out= 1015)(deflated 60%)
adding: TopNReducer.java(in = 1067) (out= 441)(deflated 58%)
adding: utils/(in = 0) (out= 0)(stored 0%)
adding: utils/MiscUtils.java(in = 687) (out= 306)(deflated 55%)
```

```
root@hp:~/topn# hadoop jar topn.jar TopN /topn_input /topn_output
2026-04-22 18:43:14,427 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-04-22 18:43:14,508 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2026-04-22 18:43:14,508 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2026-04-22 18:43:14,580 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your
application with ToolRunner to remedy this.
2026-04-22 18:43:14,656 INFO input.FileInputFormat: Total input files to process : 1
2026-04-22 18:43:14,682 INFO mapreduce.JobSubmitter: number of splits:1
2026-04-22 18:43:14,838 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local2110790824_0001
2026-04-22 18:43:14,839 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-04-22 18:43:14,928 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
2026-04-22 18:43:14,928 INFO mapreduce.Job: Running job: job_local2110790824_0001
2026-04-22 18:43:14,930 INFO mapred.LocalJobRunner: OutputCommitter set in config null
2026-04-22 18:43:14,936 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-04-22 18:43:14,936 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-04-22 18:43:14,937 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore cleanup fa
ilures: false
2026-04-22 18:43:14,937 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapreduce.lib.output.FileOutputCommitter
2026-04-22 18:43:14,963 INFO mapred.LocalJobRunner: Waiting for map tasks
2026-04-22 18:43:14,963 INFO mapred.LocalJobRunner: Starting task: attempt_local2110790824_0001_m_000000_0
2026-04-22 18:43:14,980 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-04-22 18:43:14,980 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-04-22 18:43:14,980 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore cleanup fa
ilures: false
2026-04-22 18:43:14,995 INFO mapred.Task:  Using ResourceCalculatorProcessTree : [ ]
2026-04-22 18:43:14,998 INFO mapred.MapTask: Processing split: file:/topn_input/sample.txt:0+67
2026-04-22 18:43:15,087 INFO mapred.MapTask: (EQUATOR) 0 kvi 26214396(104857584)
```

```
root@hp:~/topn# hdfs dfs -cat /topn_output/part-r-00000
spark   3
hadoop  3
data    2
big     2
mapreduce       1
root@hp:~/topn#
```